

LAPORAN PRAKTIKUM 10
SINGLE LINKED LIST



NAMA : FADILA MAYAMASITA

NIM : 254107020012

KELAS : 1C TEKNIK INFORMATIKA

SARJANA TERAPAN TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG 2026

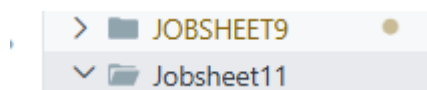
2. PRAKTIKUM

2.1 PEMBUATAN SINGLE LINKED LIST

WAKTU PERCOBAAN : 30 MENIT

DIDALAM PRAKTIKUM INI, KITA AKAN MEMPRAKTEKKAN BAGAIMANA MEMBUAT SINGLE LINKED LIST DENGAN REPRESENTASI DATA BERUPA NODE, PENGAKSESAN LINKED LIST DAN METODE PENAMBAHAN DATA.

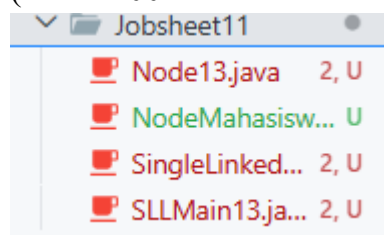
1. PADA PROJECT YANG SUDAH DIBUAT PADA MINGGU SEBELUMNYA, BUAT FOLDER BARU DENGAN NAMA JOBSHEET11



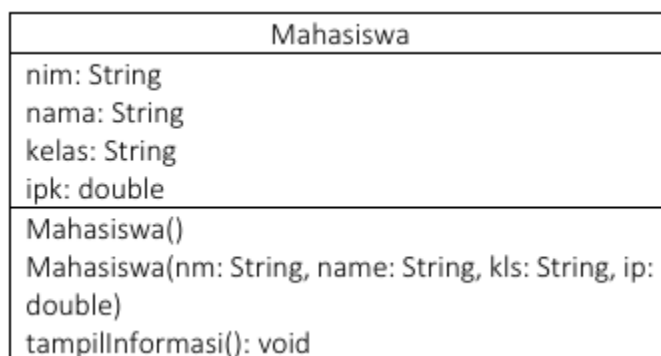
2. TAMBAHKAN CLASS-CLASS BERIKUT:

- A. MAHASISWA00.JAVA
- B. NODE00.JAVA
- C. SINGLELINKEDLIST00.JAVA
- D. SLLMAIN00.JAVA

(GANTI 00 DENGAN NOMOR PRESENSI ANDA)



3. IMPLEMENTASIKAN CLASS MAHASISWA00 SESUAI DENGAN DIAGRAM CLASS BERIKUT INI :



```
public class Mahasiswa13 {  
    String nama;  
    String nim;  
    String kelas;  
    double ipk;  
  
    public Mahasiswa13(String nim, String nama, String kelas, double ipk) {  
        this.nim = nim;  
        this.nama = nama;  
        this.kelas = kelas;  
        this.ipk = ipk;  
    }  
  
    public void tampilInformasi() {  
        System.out.println(nama + "\t" + nim + "\t" + kelas + "\t" + ipk);  
    }  
}
```

4. IMPLEMENTASI CLASS NODE

```
public class NodeMahasiswa13 {  
    Mahasiswa13 data;  
    NodeMahasiswa13 next;  
  
    public NodeMahasiswa13(Mahasiswa13 data, NodeMahasiswa13 next) {  
        this.data = data;  
        this.next = next;  
    }  
}
```

5. TAMBAHKAN ATTRIBUTE HEAD DAN TAIL PADA CLASS SINGLELINKEDLIST

```
public class SingleLinkedList00 {  
    NodeMahasiswa13 head;  
    NodeMahasiswa13 tail;  
}
```

6. SEBAGAI LANGKAH BERIKUTNYA, AKAN DIIMPLEMENTASIKAN METHOD-METHOD YANG TERDAPAT PADA SINGLELINKEDLIST :

7. TAMBAHKAN METHOD isEmpty().

```
boolean isEmpty() {  
    return (head == null);  
}
```

8. IMPLEMENTASI METHOD UNTUK MENCETAK DENGAN MENGGUNAKAN PROSES TRAVERSE.

```
public void print() {  
    if (!isEmpty()) {  
        NodeMahasiswa13 tmp = head;  
        System.out.print(s: "Isi Linked List:\t");  
        while (tmp != null) {  
            tmp.data.tampilInformasi();  
            tmp = tmp.next;  
        }  
        System.out.println(x: "");  
    } else {  
        System.out.println(x: "Linked List Kosong");  
    }  
}
```

9. IMPLEMENTASIKAN METHOD ADDFIRST().

```
public void addFirst(Mahasiswa13 input) {
    NodeMahasiswa13 ndInput = new NodeMahasiswa13(input, next: null);
    if (isEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        ndInput.next = head;
        head = ndInput;
    }
}
```

10. IMPLEMENTASIKAN METHOD ADDLAST().

```
public void addLast(Mahasiswa13 input) {
    NodeMahasiswa13 ndInput = new NodeMahasiswa13(input, next: null);
    if (isEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        tail.next = ndInput;
        tail = ndInput;
    }
}
```

11. IMPLEMENTASIKAN METHOD INSERTAFTER, UNTUK MEMASUKKAN NODE YANG MEMILIKI DATA INPUT SETELAH NODE YANG MEMILIKI DATA KEY.

```
public void insertAfter(String key, Mahasiswa13 input) {
    NodeMahasiswa13 ndInput = new NodeMahasiswa13(input, next: null);
    NodeMahasiswa13 temp = head;
    do {
        if (temp.data.nama.equalsIgnoreCase(key)) {
            ndInput.next = temp.next;
            temp.next = ndInput;
            if (ndInput.next == null) {
                tail = ndInput;
            }
            break;
        }
        temp = temp.next;
    } while (temp != null);
}
```

12. TAMBAHKAN METHOD PENAMBAHAN NODE PADA INDEKS TERTENTU.

```
,
    public void insertAt(int index, Mahasiswa13 input) {
        if (index < 0) {
            System.out.println(x: "indeks salah");
        } else if (index == 0) {
            addFirst(input);
        } else {
            NodeMahasiswa13 temp = head;
            for (int i = 0; i < index - 1; i++) {
                temp = temp.next;
            }
            temp.next = new NodeMahasiswa13(input, temp.next);
            if (temp.next.next == null) {
                tail = temp.next;
            }
        }
    }
}
```

13. PADA CLASS SLLMAIN00, BUATLAH FUNGSI MAIN, KEMUDIAN BUAT OBJECT DARI CLASS SINGLELINKEDLIST.

14. BUAT EMPAT OBJECT MAHASISWA DENGAN NAMA MHS1, MHS2, MHS3, MHS4 KEMUDIAN ISI DATA SETIAP OBJECT MELALUI KONSTRUKTOR.

```
// 1. Membuat objek mahasiswa
Mahasiswa13 mhs1 = new Mahasiswa13(nim: "24212200", nama: "Alvaro", kelas: "1A", ipk: 4.0);
Mahasiswa13 mhs2 = new Mahasiswa13(nim: "23212201", nama: "Bimon", kelas: "2B", ipk: 3.8);
Mahasiswa13 mhs3 = new Mahasiswa13(nim: "22212202", nama: "Lintia", kelas: "3C", ipk: 3.5);
Mahasiswa13 mhs4 = new Mahasiswa13(nim: "21212203", nama: "Dirga", kelas: "4D", ipk: 3.6);
```

15. TAMBAHKAN METHOD PENAMBAHAN DATA DAN PENCETAKAN DATA DI SETIAP PENAMBAHANNYA AGAR TERLIHAT PERUBAHANNYA.

```
,
    // 2. Pemanggilan method linked list
    sll.print();
    sll.addFirst(mhs4);
    sll.print();
    sll.addLast(mhs1);
    sll.print();
    sll.insertAfter(key: "Dirga", mhs3);
    sll.insertAt(index: 2, mhs2);
    sll.print();
}
```

2.1.1 VERIFIKASI HASIL PERCOBAAN

```
in Jobsheet11.5LLMain15
Linked list kosong
Isi Linked List:
Dirga 21212203 4D 3.6

Isi Linked List:
Dirga 21212203 4D 3.6
Alvaro 24212200 1A 4.0

Isi Linked List:
Dirga 21212203 4D 3.6
Cintia 22212202 3C 3.5
Bimon 23212201 2B 3.8
Alvaro 24212200 1A 4.0
```

2.1.2 PERTANYAAN

1. MENGAPA HASIL COMPILE KODE PROGRAM DI BARIS PERTAMA MENGHASILKAN “LINKED LIST KOSONG”?

JAWABAN :

KARENA PADA SAAT PERINTAH DICETAK TERSEBUT DIPANGGIL, HASILNYA BELUM ADA DATA SAMA SEKALI YANG DIMASUKKAN ATAU DISAMBUNGAN KE DALAM STRUKTUR LINKED LIST.

2. JELASKAN KEGUNAAN VARIABLE TEMP SECARA UMUM PADA SETIAP METHOD!

JAWABAN :

1. MENJAGA POINTER UTAMA (HEAD) AGAR TIDAK HILANG
LINKED LIST ITU DIKENDALIKAN OLEH VARIABEL HEAD. JIKA LANGSUNG MENGGESER VARIABEL HEAD BUAT BACA DATA, AKAN KEHILANGAN REFERENSI KE BAGIAN AWAL LIST. SEHINGGA DATA-DATA SEBELUMNYA TIDAK DAPAT DIAKSES ATAU BAHKAN TERHAPUS DARI MEMORI. MAKA HEAD AKAN DISALIN DULU KE TEMP SEHINGGA TEMP YANG BEKERJA BUAT MENGGESER, SEDANGKAN HEAD TETAP AMAN BERADA DI POSISINYA
2. SEBAGAI ALAT BANTU PENELUSURAN
TEMP DIPAKAI DALAM PERULANGAN BUAT MEMINDAHKAN FOKUS DARI SATU NODE KE NODE SETELAHNYA, PADA METHOD PRINT() : TEMP BEGERAK DARI DEPAN SAMPAI UJUNG AKHIR BUAT MENGEKSTRAK DAN MENAMPILKAN INFORMASI DARI MASING-MASING NODE. PADA METHOD INSERTAT() : TEMP DIPAKAI SEBAGAI PENANDA LANGKAH BUAT BERHENTI TEPAT DI POSISI SATU NODE INDEKS TARGET AGAR BISA MEMANIPULASI SAMBUNGAN PRINTER
3. MEMBANTU PENCARIAN DATA

PADA METHOD INSERTAFTER() : TEMP BERFUNGSI BUAT PEMERIKSA. TEMP BAKAL MEMBANDINGKAN DATA PADA NODE TEMPAT IIA BERDIRI SAAT ITU DENGAN DATA/KUNCI YANG DI CARI.

3. LAKUKAN MODIFIKASI AGAR DATA DAPAT DITAMBAHKAN DARI KEYBOARD!

JAWABAN:

```
1 package Jobsheet11;
2 import java.util.Scanner;
3
4 public class SLLMain13 {
    Run | Debug | Run main | Debug main
5     public static void main(String[] args) {
6         SingleLinkedList13 sll = new SingleLinkedList13();
7         Scanner sc = new Scanner(System.in);
8
9         System.out.println(x: "--- Input Data Mahasiswa ke linked list ---");
10        System.out.println(x: "Masukkan jumlah mahasiswa yang ingin dininput: ");
11        int jumlah = sc.nextInt();
12        sc.nextInt();
13
14        for (int i = 0; i < jumlah; i++) {
15            System.out.println("\nData Mahasiswa ke-" + (i + 1));
16            System.out.print(s: "NIM   : ");
17            String nim = sc.nextLine();
18            System.out.println(x: "Nama   : ");
19            String nama = sc.nextLine();
20            System.out.println(x: "Kelas : ");
21            String kelas = sc.nextLine();
22            System.out.println(x: "IPK   : ");
23            double ipk = sc.nextDouble();
24            sc.nextLine();
25
26            // membuat objek mahasiswa berdasarkan input keyboard
27            Mahasiswa13 mhs = new Mahasiswa13(nim, nama, kelas, ipk);
28
29            // memasukkan data ke linked list
30            sll.addLast(mhs);
31
32        }
33
34        System.out.println(x: "\n--- Hasil Traverser linked list ---");
35        sll.print();
36
37        sc.close();
38    }
39 }
```

2.2 MODIFIKASI ELEMEN PADA SINGLE LINKED LIST

WAKTU PERCOBAAN : 30 MENIT

DIDALAM PRAKTIKUM INI, KITA AKAN MEMPRAKTEKKAN BAGAIMANA MENGAkses ELEMEN, MENDAPATKAN INDEKS DAN MELAKUKAN PENGHAPUSAN DATA PADA SINGLE LINKED LIST.:

2.2.1 LANGKAH-LANGKAH PERCOBAAN

1. IMPLEMENTASIKAN METHOD UNTUK MENGAkses DATA DAN INDEKS PADA LINKED LIST
2. TAMBAHKAN METHOD UNTUK MENDAPATKAN DATA PADA INDEKS TERTENTU PADA CLASS SINGLE LINKED LIST

```
public void getData(int index) {
    if (isEmpty()) {
        System.out.println(x: "Linked List Kosong");
    } else {
        NodeMahasiswa13 tmp = head;

        for (int i = 0; i < index; i++) {
            if (tmp.next == null) {
                System.out.println(x: "Indeks di luar jangkauan");
                return;
            }
            tmp = tmp.next;
        }
        tmp.data.tampilInformasi();
    }
}
```

3. IMPLEMENTASIKAN METHOD INDEXOF.

```
public int indexOf(String key) {
    NodeMahasiswa13 tmp = head;
    int index = 0;

    while (tmp != null && !tmp.data.nama.equalsIgnoreCase(key)) {
        tmp = tmp.next;
        index++;
    }

    if (tmp == null) {
        return -1;
    } else {
        return index;
    }
}
```


4. TAMBAHKAN METHOD REMOVEFIRST PADA CLASS SINGLELINKEDLIST

```
public void removeFirst() {
    if (isEmpty()) {
        System.out.println(x: "Linked list masih kosong, tidak dapat dihapus");
    } else if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
    }
}
```

5. TAMBAHKAN METHOD UNTUK MENGHAPUS DATA PADA BAGIAN BELAKANG PADA CLASS SINGLELINKEDLIST

```
public void removeLast() {
    if (isEmpty()) {
        System.out.println(x: "Linked list masih kosong, tidak dapat dihapus");
    } else if (head == tail) {
        head = tail = null;
    } else {
        NodeMahasiswa13 temp = head;
        while (temp.next != tail) {
            temp = temp.next;
        }
        temp.next = null;
        tail = temp;
    }
}
```

6. SEBAGAI LANGKAH BERIKUTNYA, AKAN DIIMPLEMENTASIKAN METHOD REMOVE

```
public void remove(String key) {
    if (isEmpty()) {
        System.out.println(x: "Linked list masih kosong, tidak dapat dihapus!");
    } else {
        NodeMahasiswa13 temp = head;
        while (temp != null) {
            if (temp.data.nama.equalsIgnoreCase(key)){
                this.removeFirst();
                break;
            } else if (temp.data.nama.equalsIgnoreCase(key)) {
                temp.next = temp.next.next;
                if (temp.next == null) {
                    tail = temp;
                }
                break;
            }
            temp = temp.next;
        }
    }
}
```

7. IMPLEMENTASI METHOD UNTUK MENGHAPUS NODE DENGAN MENGGUNAKAN INDEX.

```
public void removeAt(int index) {
    if (index == 0) {
        removeFirst();
    } else {
        NodeMahasiswa13 temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = temp.next.next;
        if (temp.next == null) {
            tail = temp;
        }
    }
}
```

8. KEMUDIAN, COBA LAKUKAN PENGAKSESAN DAN PENGHAPUSAN DATA DI METHOD MAIN PADA CLASS SLLMAIN DENGAN MENAMBAHKAN KODE BERIKUT

```
System.out.println(x: "data index 1 : ");
sll.getData(index: 1);

System.out.println("data mahasiswa an Bimon berada pada index : "+sll.indexOf(key: "bimon"));
System.out.println();

sll.removeFirst();
sll.removeLast();
sll.print();
sll.removeAt(index: 0);
sll.print();
```

9. JALANKAN CLASS SLLMAIN

2.2.2 VERIFIKASI HASIL PERCOBAAN

```
data index 1 :
Cintia  22212202      3C      3.5
data mahasiswa an Bimon berada pada index : 2

Isi Linked List:
Cintia  22212202      3C      3.5
Bimon   23212201      2B      3.8

Isi Linked List:
Bimon   23212201      2B      3.8
```

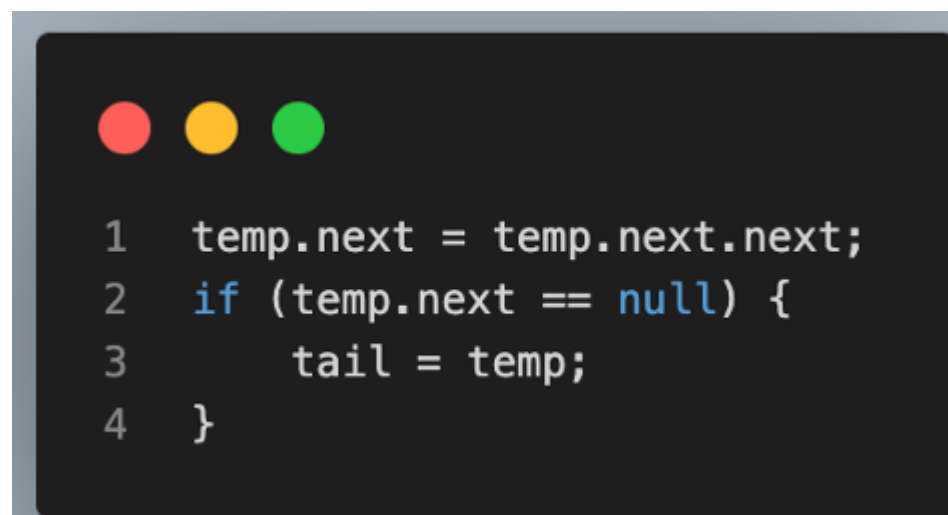
2.2.3 PERTANYAAN

1. MENGAPA DIGUNAKAN KEYWORD BREAK PADA FUNGSI REMOVE? JELASKAN!

JAWABAN:

BREAK PADA FUNGSI REMOVE DIPAKAI BUAT LANGSUNG MENGHENTIKAN PERULANGAN PENULUSURAN SETELAH DATA MAHASISWA YANG DICARI BERHASIL DITEMUKAN DAN DI HAPUS. JIKA DATA TARGET SUDAH BERHASIL DIHAPUS, PROGRAM TIDAK PERLU LAGI MELANJUTKAN PENCARIAN SAMPAI AKHIR LINKED LIST. KARENA BUAT MENGHEMAT WAKTU EKSEKUSI PROGRAM SECARA PROSES PENCARIAN DIHENTIKAN SECARA EFISIEN. SELAIN ITU, BREAK BERFUNGSI BUAT MENCEGAH TERJADINYA ERROR AKHIBAT PROGRAM MENCOBA AKSES NODE KOSONG SETELAH STRUKTUR POINTER BERUBAH.

2. JELASKAN KEGUNAAN KODE DIBAWAH PADA METHOD REMOVE



```
1  temp.next = temp.next.next;
2  if (temp.next == null) {
3      tail = temp;
4  }
```

JAWABAN :

BARIS PERTAMA YAITU `TEMP.NEXT = TEMP.NEXT.NEXT` ; BUAT MENGHAPUS ELEMEN TARGET DENGAN CARA MELOMPATI POINTER ELEMEN TERSEBUT DAN LANGSUNG MENYAMBUNGAN KE ELEMEN SETELAHNYA. SEDANGKAN `IF (TEMP. NEXT == NULL)` BUAT MEMERIKSA APAKAH ELEMEN YANNG BARU SAJA DIHAPUS BERADA DI POSISI PALING BELAKANG ATAU UJUNG AKHIR. PERINTAH `TAIL = TEMP`; BAKAL DIJALANKAN BUAT MEMINDAHKAN PENUNJUK TAIL KE KODE TEMP SAAT INI.

3. TUGAS

WAKTU Pengerjaan : 50 MENIT

BUATLAH IMPLEMENTASI PROGRAM ANTRIAN LAYANAN UNIT KEMAHASISWAAN SESUAI DENGAN BERIKUT INI :

- A. IMPLEMENTASI ANTRIAN MENGGUNAKAN QUEUE BERBASIS LINKED LIST!
- B. PROGRAM MERUPAKAN PROYEK BARU BUKAN MODIFIKASI DARI PERCOBAAN
- C. KETIKA SEORANG MAHASISWA AKAN MENGANTRI, MAKA DIA HARUS MENDAFTARKAN DATANYA
- D. CEK ANTRIAN KOSONG, CEK ANTRIAN PENUH, MENGOSONGKAN ANTRIAN.
- E. MENAMBAHKAN ANTRIAN
- F. MEMANGGIL ANTRIAN
- G. MENAMPILKAN ANTRIAN TERDEPAN DAN ANTRIAN PALING AKHIR
- H. MENAMPILKAN JUMLAH MAHASISWA YANG MASIH MENGANTRE.

JAWABAN :

1. MEMBUAT FILE MAHASISWA TERLEBIH DAHULU

FILE INI DIPAKAI BUAT SIMPAN BLUEPRINT DATA MAHASISWA YANG MENGANTRI

```
1 package J005Sheet11,
2
3 public class Mahasiswa213 {
4     String nim;
5     String nama;
6     String keperluan;
7
8     // konstruktor buat mendaftar data mahasiswa
9     public Mahasiswa213(String nim, String nama, String keperluan) {
10         this.nim = nim;
11         this.nama = nama;
12         this.keperluan = keperluan;
13     }
14
15     // method buat menampilkan informasi mahasiswa
16     public void tampilkanData() {
17         System.out.println("NIM          : " + nim);
18         System.out.println("NAMA          : " + nama);
19         System.out.println("KEPERLUAN     : " + keperluan);
20     }
21 }
22 }
```

2. MEMBUAT FILE NODE

FILE NODE BERTINDAK SEBAGAI WADAH YANG MENYIMPAN OBJEK MAHASISWA SEKALIGUS NEXT BUAT MENYAMBUNGAN ANTRIAN

```
public class NodeMahasiswa213 {  
    Mahasiswa213 data;  
    NodeMahasiswa213 next;  
  
    public NodeMahasiswa213(Mahasiswa213 data) {  
        this.data = data;  
        this.next = next;  
    }  
}
```

3. MEMBUAT FILE QUEUE LINKED LIST

CLASS INI BERISI KODE QUEUE YANG BERBASIS LINKED LIST BUAT MENGELOLA ANTRIAN LAYANAN MAHASISWA

```
public class QueueLinkedList13 {  
    private NodeMahasiswa213 head;  
    private NodeMahasiswa213 tail;  
    private int size;  
    private int maxCapacity;  
  
    public QueueLinkedList13(int maxCapacity) {  
        this.head = null;  
        this.tail = null;  
        this.size = 0;  
        this.maxCapacity = maxCapacity;  
    }  
  
    // method cek antrian kosong (queue)  
    public boolean isEmpty() {  
        if (size == 0) {  
            return true;  
        } else {  
            return false;  
        }  
    }  
}
```

METHOD ISEMPTY() BERFUNGSI UNTUK MENGECEK APAKAH QUEUE SEDANG KOSONG ATAU TIDAK DENGAN MEMERIKSA VARIABEL SIZE. JIKA NILAI SIZE SAMA DENGAN 0 MAKA METHOD AKAN MENGEMBALIKAN NILAI FALSE(TERISI).

```
// method cek antrian penuh (queue)
public boolean isFull() {
    if (size == maxCapacity) {
        return true;
    } else {
        return false;
    }
}
```

METHOD ISFULL() BERFUNGSI BUAT MENGECEK APAKAH ANTREAN SUDAH PENUH ATAU BELUM DENGAN MEMBANDINGKAN NILAI SIZE DAN MAXCAPACITY. JIKA NILAI KEDUAYA SAMA, MAKA AKAN MENGEMBALIKAN NILAI (TRUE) DAN JIKA TIDAK SAMA AKAN MENGEMBALIKAN (FALSE)

```
// method mengosongkan antrian (queue)
public void clear() {
    head = null;
    tail = null;
    size = 0;
    System.out.println(x: "Antrian telah dikosong!");
}
```

METHOD CLEAR() BERFUNGSI BUAT MENGOSONGKAN SELURUH ISI ANTREAN SECARA INSTAN DENGAN MENGUBAH POINTER HEAD DAN TAIL MENJADI NULL.

```
// method menambahkan antrian (Enqueue)
public void enqueue(Mahasiswa213 mhs) {
    if(isFull()) {
        System.out.println(x: "Maaf antrian penuh tidak dapat menambahkan data saat ini!");
        return;
    }
    NodeMahasiswa213 newNode = new NodeMahasiswa213(mhs);
    if (isEmpty()) {
        head = newNode;
        tail = newNode;
    } else {
        tail.next = newNode;
        tail = newNode;
    }
    size++;
    System.out.println(mhs.nama + "Berhasil masuk ke dalam antrian");
}
```

METHOD ENQUEUE BERFUNGSI BUAT MENAMBAHKAN DATA MAHASISWA BARU KE POSISI PALING BELAKANG SETELAH MEMASTIKAN ANTREAN BELUM PENUH, JIKA ANTREAN KOSONG, DATA BARU MENJADI HEAD SEKALIGUS

TAIL, NAMUN JIKA SUDAH BERISI, DATA DISAMBUNGGAN DI BELAKANG TAIL LAMA , LALU POINTER TAIL DIPERBARUI DAN JUMLAH SIZE DITAMBAH.

```
// method memanggil / mengeluarkan antrian terdepan (dequeue)
public void dequeue() {
    if (isEmpty()) {
        System.out.println(x: "Antrian kosong! tidak ada mahasiswa yang dapat dipanggil");
        return;
    }

    Mahasiswa213 mhsDipanggil = head.data;
    head = head.next;
    size--;

    if (head == null) {
        tail = null;
    }
    System.out.println(x: "--- Memanggil Antrian ---");
    mhsDipanggil.tampilkanData();
    System.out.println(x: "Silahkan menuju ke loket unit kemahasiswaan ");
}
```

METHOD DEQUEUE BERFUNGSI BUAT MENGELUARKAN DAN MEMPROSES DATA MAHASISWA YANG BERADA DI ANTREAN PALING DEPAN SETELAH MEMSTIKAN ANTREAN KOSONG.

```
// method menampilkan jumlah mahasiswa yang masih mengantri
public int getSize() {
    return size;
}
```

METHOD GETSIZE() BERFUNGSI SEBAGAI FUNGSI BETTER UNTUK MENGAMBIL DAN MENGEMBALIKAN JUMLAH TOTAL MAHASISWA YANG SAAT INI BERADA DI DALAM ANTREN.

```
// method menampilkan seluru isi antrian saat ini
public void print() {
    if (isEmpty()) {
        System.out.println(x: "Antrian saat ini masih kosong");
        return;
    }
    System.out.println(x: "\n---DAFTAR MAHASISWA DALAM ANTRIAN---");
    NodeMahasiswa213 current = head;
    int no = 1;
    while (current != null) {
        System.out.println("No. Antrian: " + no);
        current.data.tampilkanData();
        System.out.println(x: "-----");
        current = current.next;
        no++;
    }
}
```

METHOD INI BERFUNGSI BUAT MENAMPILKAN DATA MAHASISWA YANG BERADA DI POSISI PALING DEPAN DAN PALING BELAKANG SECARA BERSAMAAN SETELAH MEMASTIKAN ANTREAN TIDAK KOSONG.

4. MEMBUAT FILE MAINANTRIAN

FILE INI BERFUNGSI SEBAGAI ANTARMUKA MENU INTERAKTIF BERBASIS TEKS AGAR PENGGUNA DAPAT MENGOPERASIKAN SISTEM ANTRIAN DARI KEYBOARD.

```
Jobsheet11 > AntrianMain13.java > Language Support for Java(TM) by Red Hat > AntrianMain13 > main(String[])
1  package Jobsheet11;
2  import java.util.Scanner;
3
4  public class AntrianMain13 {
5      Run | Debug | Run main | Debug main
6      public static void main(String[] args) {
7          Scanner sc = new Scanner(System.in);
8          QueueLinkedList13 antrianLayan = new QueueLinkedList13(maxCapacity: 5);
9          int pilihan;
10
11          do {
12              System.out.println(x: "\n----- Sistem antrian layanan mahasiswa -----");
13              System.out.println(x: "1. Daftarkan mahasiswa ke antrian");
14              System.out.println(x: "2. Panggil/layani antrian terdepan");
15              System.out.println(x: "3. Cek jumlah mahasiswa yang mengantre");
16              System.out.println(x: "4. Lihat antrian terdepan & paling akhir");
17              System.out.println(x: "5. Tampilkan seluruh daftar antrian");
18              System.out.println(x: "6. Cek status antrian (kosong/penuh)");
19              System.out.println(x: "7. Kosongkan seluruh antrian");
20              System.out.println(x: "8. keluar program");
21              System.out.print(s: "Pilih Menu (1-8): ");
22              pilihan = sc.nextInt();
23              sc.nextLine();
24
25          switch (pilihan) {
26              case 1:
27                  if (antrianLayan.isFull()) {
28                      System.out.println(x: "Antrian sudah penuh! silahkan tunggu beberapa saat");
29                  } else {
30                      System.out.println(x: "\n--- Pendaftaran data mahasiswa ---");
31                      System.out.print(s: "NIM          : ");
32                      String nim = sc.nextLine();
33                      System.out.print(s: "Nama          : ");
34                      String nama = sc.nextLine();
35                      System.out.print(s: "Keperluan     : ");
36                      String keperluan = sc.nextLine();
37
38                      Mahasiswa213 mhsBaruMahasiswa213 = new Mahasiswa213(nim, nama, keperluan);
39                      antrianLayan.enqueue(mhsBaruMahasiswa213);
40                  }
41                  break;
42
43              case 2:
44                  antrianLayan.dequeue();
45                  break;
46
47              case 3:
48                  System.out.println("Jumlah mahasiswa yang masih mengantre: " + antrianLayan.getSize() + " orang.");
49                  break;
50
51              case 4:
52                  antrianLayan.tampilTerdepanDanTerakhir();
53                  break;
54
55              case 5:
56                  antrianLayan.print();
57                  break;
58
59              default:
60                  break;
61          }
62      }
63  }
```



```

        break;
    case 6:
        System.out.println("Apakah antrian kosong? " + (antrianLayan.isEmpty() ? "Ya" : "Tidak"));
        System.out.println("Apakah antrian penuh? " + (antrianLayan.isFull() ? "Ya" : "Tidak"));
        break;
    case 7:
        System.out.print(s: "Apakah ingin mengosongkan antrian? (y/n): ");
        String konfirmasi = sc.nextLine();
        if (konfirmasi.equalsIgnoreCase(anotherString: "y")) {
            antrianLayan.clear();
        } else {
            System.out.println(x: "Pengosongan antrian dibatalkan.");
        }
        break;
    case 8:
        System.out.println(x: "Terima kasih! telah menggunakan program ini");
        break;

    default:
        System.out.println(x: "Pilihan tidak valid silahkan masukan angka 1-8");
}
} while (pilihan != 8);

sc.close();

```

1. MENDAFTARKAN MAHASISWA KE ANTRIAN (MENU 1)

```

----- Sistem antrian layanan mahasiswa -----
1. Daftrakan mahasiswa ke antrian
2. Panggil/layani antrian terdepan
3. Cek jumlah mahasiswa yang mengantre
4. Lihat antrian terdepan & paling akhir
5. Tampilkan seluruh daftar antrian
6. Cek status antrian (kosong/penuh)
7. Kosongkan seluruh antrian
8. keluar program
Pilih Menu (1-8): 1

```

```

--- Pendaftaran data mahaasiswa ---
NIM          : 242122000
Nama         : Alvaro
Keperluan    : Legalisir Ijazah
AlvaroBerhasil masuk ke dalam antrian

```

Pilih Menu (1-8): 1

```

--- Pendaftaran data mahaasiswa ---
NIM          : 245367654
Nama         : Bimon
Keperluan    : Legalisir ijazah
BimonBerhasil masuk ke dalam antrian

```

```
--- Pendaftaran data mahaasiswa ---  
NIM           : 241366754  
Nama          : Cintia  
Keperluan     : Mengambil KTM  
CintiaBerhasil masuk ke dalam antrian
```

2. MENAMPILKAN SELURUH DAFTAR ANTRIAN

```
----- Sistem antrian layanan mahasiswa -----  
1. Daftarkan mahasiswa ke antrian  
2. Panggil/layani antrian terdepan  
3. Cek jumlah mahasiswa yang mengantre  
4. Lihat antrian terdepan & paling akhir  
5. Tampilkan seluruh daftar antrian  
6. Cek status antrian (kosong/penuh)  
7. Kosongkan seluruh antrian  
8. keluar program  
Pilih Menu (1-8): 5
```

```
---DAFTAR MAHASISWA DALAM ANTRIAN---  
No. Antrian: 1  
NIM           : 242122000  
NAMA          : Alvaro  
KEPERLUAN     : Legalisir Ijazah  
-----  
No. Antrian: 2  
NIM           : 245367654  
NAMA          : Bimon  
KEPERLUAN     : Legalisir ijazah  
-----  
No. Antrian: 3  
NIM           : 241366754  
NAMA          : Cintia  
KEPERLUAN     : Mengambil KTM  
-----
```

3. MEMANGGIL/MENGELUARKAN ANTRIAN TERDEPAN

```
----- Sistem antrian layanan mahasiswa -----
1. Daftarkan mahasiswa ke antrian
2. Panggil/layani antrian terdepan
3. Cek jumlah mahasiswa yang mengantre
4. Lihat antrian terdepan & paling akhir
5. Tampilkan seluruh daftar antrian
6. Cek status antrian (kosong/penuh)
7. Kosongkan seluruh antrian
8. keluar program
Pilih Menu (1-8): 2
--- Memanggil Antrian ---
NIM          : 242122000
NAMA         : Alvaro
KEPERLUAN    : Legalisir Ijazah
Silahkan menuju ke loket unit kemahasiswaan
```

4. MENAMPILKAN ANTRIAN TERDEPAN DAN PALING AKHIR

```
----- Sistem antrian layanan mahasiswa -----
1. Daftarkan mahasiswa ke antrian
2. Panggil/layani antrian terdepan
3. Cek jumlah mahasiswa yang mengantre
4. Lihat antrian terdepan & paling akhir
5. Tampilkan seluruh daftar antrian
6. Cek status antrian (kosong/penuh)
7. Kosongkan seluruh antrian
8. keluar program
Pilih Menu (1-8): 4
-----
[ANTRIAN TERDEPAN]
NIM          : 245367654
NAMA         : Bimon
KEPERLUAN    : Legalisir ijazah
[ANTREAN BELAKANG]
NIM          : 241366754
NAMA         : Cintia
KEPERLUAN    : Mengambil KTM
```